

8. 联合查询

1. 本节目标

- 了解联合查询的计算过程
- 掌握内连接、左外连接、右外连和自连接接查询
- 掌握子查询和合并查询
- 掌握插入查询结果和根据查询结果创建表

2. 简介

2.1 为什么要使用联合查询

在数据设计时由于范式的要求，数据被拆分到多个表中，那么要查询一个条数据的完整信息，就要从多个表中获取数据，如下图所示：要获取学生的基本信息和班级信息就要从学生表和班级表中获取，这时就需要使用联合查询，这里的联合指的是多个表的组合。

学生表

编号	姓名	性别	班级编号
1	张三	男	1
2	李四	女	1
3	王五	男	2

班级表

编号	班级
1	java113
2	java78
3	C++110

2.2 多表联合查询时MYSQL内部是如何进行计算的

- 参与查询的所有表取笛卡尔积，结果集在临时表中

编号	姓名	性别	班级编号	编号	班级
1	张三	男	1	1	java113
1	张三	男	1	2	java78
1	张三	男	1	3	C++110
2	李四	女	1	1	java113
2	李四	女	1	2	java78
2	李四	女	1	3	C++110
3	王五	男	2	1	java113
3	王五	男	2	2	java78
3	王五	男	2	3	C++110

- 观察哪些记录是有效数据，根据两个表的关联关系过滤掉无效数据

编号	姓名	性别	班级编号	编号	班级
1	张三	男	1	1	java113
1	张三	男	1	2	java78
1	张三	男	1	3	C++110
2	李四	女	1	1	java113
2	李四	女	1	2	java78
2	李四	女	1	3	C++110
3	王五	男	2	1	java113
3	王五	男	2	2	java78
3	王五	男	2	3	C++110

如果联合查询表的个数越多，表中的数据量越大，临时表就会越大，所以根据实际情况确定联合查询表的个数

2.3 构造练习案例数据

```
1 # 课程表
2 insert into course (name) values ('Java'), ('C++'), ('MySQL'), ('操作系统'), ('计
   算机网络'), ('数据结构');
3
4 # 班级表
5 insert into class(name) values ('Java001班'), ('C++001班'), ('前端001班');
6
7 # 学生表
8 insert into student (name, sno, age, gender, enroll_date, class_id) values
9 ('唐三藏', '100001', 18, 1, '1986-09-01', 1),
10 ('孙悟空', '100002', 18, 1, '1986-09-01', 1),
11 ('猪悟能', '100003', 18, 1, '1986-09-01', 1),
12 ('沙悟净', '100004', 18, 1, '1986-09-01', 1),
13 ('宋江', '200001', 18, 1, '2000-09-01', 2),
14 ('武松', '200002', 18, 1, '2000-09-01', 2),
15 ('李逵', '200003', 18, 1, '2000-09-01', 2),
16 ('不想毕业', '200004', 18, 1, '2000-09-01', 2);
17
18 # 成绩表
19 insert into score (score, student_id, course_id) values
20 (70.5, 1, 1), (98.5, 1, 3), (33, 1, 5), (98, 1, 6),
21 (60, 2, 1), (59.5, 2, 5),
22 (33, 3, 1), (68, 3, 3), (99, 3, 5),
23 (67, 4, 1), (23, 4, 3), (56, 4, 5), (72, 4, 6),
24 (81, 5, 1), (37, 5, 5),
25 (56, 6, 2), (43, 6, 4), (79, 6, 6),
```

2.4 案例：一个完整的联合查询的过程

- 查询学生姓名为孙悟空的详细信息，包括学生个人信息和班级信息

1. 确定参与查询的表，学生表和班级表

```

1 # 在from后同时写所有参与查询的表，并用逗号隔开
2 mysql> select * from student, class;
3 +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
4 | id | name | sno | age | gender | enroll_date | class_id | id |
5 +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
6 | 1 | 唐三藏 | 100001 | 18 | 1 | 1986-09-01 | 1 | 3 | 前端
7 | 1 | 唐三藏 | 100001 | 18 | 1 | 1986-09-01 | 1 | 2 |
8 | 1 | 唐三藏 | 100001 | 18 | 1 | 1986-09-01 | 1 | 1 |
9 | 2 | 孙悟空 | 100002 | 18 | 1 | 1986-09-01 | 1 | 3 | 前端
10 | 2 | 孙悟空 | 100002 | 18 | 1 | 1986-09-01 | 1 | 2 |
11 | 2 | 孙悟空 | 100002 | 18 | 1 | 1986-09-01 | 1 | 1 |
12 | 3 | 猪悟能 | 100003 | 18 | 1 | 1986-09-01 | 1 | 3 | 前端
13 | 3 | 猪悟能 | 100003 | 18 | 1 | 1986-09-01 | 1 | 2 |
14 | 3 | 猪悟能 | 100003 | 18 | 1 | 1986-09-01 | 1 | 1 |
15 | 4 | 沙悟净 | 100004 | 18 | 1 | 1986-09-01 | 1 | 3 | 前端
16 | 4 | 沙悟净 | 100004 | 18 | 1 | 1986-09-01 | 1 | 2 |
17 | 4 | 沙悟净 | 100004 | 18 | 1 | 1986-09-01 | 1 | 1 |
18 | 5 | 宋江 | 200001 | 18 | 1 | 2000-09-01 | 1 | 3 | 前
19 | 5 | 宋江 | 200001 | 18 | 1 | 2000-09-01 | 1 | 2 |
20 | 5 | 宋江 | 200001 | 18 | 1 | 2000-09-01 | 1 | 1 |

```

```

21 | 6 | 武松 | 200002 | 18 | 1 | 2000-09-01 | 1 | 3 | 前
    端001班 |
22 | 6 | 武松 | 200002 | 18 | 1 | 2000-09-01 | 1 | 2 |
    C++001班 |
23 | 6 | 武松 | 200002 | 18 | 1 | 2000-09-01 | 1 | 1 |
    Java001班 |
24 | 7 | 李逵 | 200003 | 18 | 1 | 2000-09-01 | 1 | 3 | 前
    端001班 |
25 | 7 | 李逵 | 200003 | 18 | 1 | 2000-09-01 | 1 | 2 |
    C++001班 |
26 | 7 | 李逵 | 200003 | 18 | 1 | 2000-09-01 | 1 | 1 |
    Java001班 |
27 | 8 | 不想毕业 | 200004 | 18 | 1 | 2000-09-01 | 1 | 3 | 前端
    001班 |
28 | 8 | 不想毕业 | 200004 | 18 | 1 | 2000-09-01 | 1 | 2 |
    C++001班 |
29 | 8 | 不想毕业 | 200004 | 18 | 1 | 2000-09-01 | 1 | 1 |
    Java001班 |
30 +-----+-----+-----+-----+-----+-----+-----+-----+
    -----+
31 24 rows in set (0.00 sec)

```

2. 确定连接条件，student表中的class_id与class表中id列的值相等

```

1 # 在where子句中加入连接条件
2 mysql> select * from student, class where student.class_id = class.id;
3 +-----+-----+-----+-----+-----+-----+-----+-----+
    -----+
4 | id | name | sno | age | gender | enroll_date | class_id | id |
   name |
5 +-----+-----+-----+-----+-----+-----+-----+-----+
    -----+
6 | 1 | 唐三藏 | 100001 | 18 | 1 | 1986-09-01 | 1 | 1 |
   Java001班 |
7 | 2 | 孙悟空 | 100002 | 18 | 1 | 1986-09-01 | 1 | 1 |
   Java001班 |
8 | 3 | 猪悟能 | 100003 | 18 | 1 | 1986-09-01 | 1 | 1 |
   Java001班 |
9 | 4 | 沙悟净 | 100004 | 18 | 1 | 1986-09-01 | 1 | 1 |
   Java001班 |
10 | 5 | 宋江 | 200001 | 18 | 1 | 2000-09-01 | 1 | 1 |
   Java001班 |
11 | 6 | 武松 | 200002 | 18 | 1 | 2000-09-01 | 1 | 1 |
   Java001班 |

```

```

12 | 7 | 李连 | 200003 | 18 | 1 | 2000-09-01 | 1 | 1 |
    Java001班 |
13 | 8 | 不想毕业 | 200004 | 18 | 1 | 2000-09-01 | 1 | 1 |
    Java001班 |
14 +-----+-----+-----+-----+-----+-----+-----+-----+
    -----+
15 8 rows in set (0.00 sec)

```

3. 加入查询条件

```

1 # 依题意添加where条件
2 mysql> select * from student, class where student.class_id = class.id and name
  = '宋江';
3 ERROR 1052 (23000): Column 'name' in where clause is ambiguous
4 # 由于两个表中都有name列，所以MySQL不清楚具体要使用哪个列，这时可以用“表名.列号”的方式指
  定具体的列
5
6 # 得到目标记录行
7 mysql> select * from student, class where student.class_id = class.id and
  student.name = '宋江';
8 +-----+-----+-----+-----+-----+-----+-----+-----+
  ---+
9 | id | name | sno | age | gender | enroll_date | class_id | id | name
  |
10 +-----+-----+-----+-----+-----+-----+-----+-----+
  ---+
11 | 5 | 宋江 | 200001 | 18 | 1 | 2000-09-01 | 1 | 1 | Java001
  班 |
12 +-----+-----+-----+-----+-----+-----+-----+-----+
  ---+
13 1 row in set (0.00 sec)

```

4. 精减查询结果字段

```

1 select
2     student.id,
3     student.name,
4     student.sno,
5     student.age,
6     student.gender,
7     student.enroll_date,
8     class.name
9 from

```

```

10     student, class
11 where
12     student.class_id = class.id
13 and
14     student.name = '宋江';
15
16 +-----+-----+-----+-----+-----+-----+-----+
17 | id | name  | sno   | age  | gender | enroll_date | name          |
18 +-----+-----+-----+-----+-----+-----+-----+
19 | 5  | 宋江  | 200001 | 18   | 1      | 2000-09-01  | Java001班    |
20 +-----+-----+-----+-----+-----+-----+-----+
21 1 row in set (0.00 sec)

```

5. 可以为表名指定别名

```

1 select
2     s.id,
3     s.name,
4     s.sno,
5     s.age,
6     s.gender,
7     s.enroll_date,
8     c.name
9 from
10     student s , class c
11 where
12     s.class_id = c.id
13 and
14     s.name = '宋江';
15
16 +-----+-----+-----+-----+-----+-----+-----+
17 | id | name  | sno   | age  | gender | enroll_date | name          |
18 +-----+-----+-----+-----+-----+-----+-----+
19 | 5  | 宋江  | 200001 | 18   | 1      | 2000-09-01  | Java001班    |
20 +-----+-----+-----+-----+-----+-----+-----+
21 1 row in set (0.00 sec)

```

3. 内连接

3.1 语法

```

1 select 字段 from 表1 别名1, 表2 别名2 where 连接条件 and 其他条件;

```

2 select 字段 from 表1 别名1 [inner] join 表2 别名2 on 连接条件 where 其他条件;

3.2 示例

- 查询"唐三藏"同学的成绩

```
1 mysql> select s.name, sc.score from student s join score sc on sc.student_id =
  s.id where s.name = '唐三藏';
2 +-----+-----+
3 | name      | score |
4 +-----+-----+
5 | 唐三藏    | 70.5  |
6 | 唐三藏    | 98.5  |
7 | 唐三藏    | 33    |
8 | 唐三藏    | 98    |
9 +-----+-----+
10 4 rows in set (0.00 sec)
```

- 查询所有同学的总成绩，及同学的个人信息

```
1 mysql> select s.name, sum(sc.score) from student s, score sc where
  sc.student_id = s.id group by (s.id);
2 +-----+-----+
3 | name      | sum(sc.score) |
4 +-----+-----+
5 | 唐三藏    | 300           |
6 | 孙悟空    | 119.5         |
7 | 猪悟能    | 200           |
8 | 沙悟净    | 218           |
9 | 宋江      | 118           |
10 | 武松      | 178           |
11 | 李逵      | 172           |
12 +-----+-----+
13 7 rows in set (0.00 sec)
```

Group by 使用了student.id进行分组，查询表列表中的student.name没有出现在Group by分组中，也没有包含在聚合函数中，这是因为SQL规定在Group by分组查询时，如果查询列表中的列没有出现在GROUP BY子句中，但这些列的值在每个分组内部是相同的，那么它们可以出现在查询结果中。

- 查询所有同学每门课的成绩，及同学的个人信息

```

1 select
2   s.id as id,
3   s.name as 姓名,
4   s.sno as 学号,
5   s.gender as 性别,
6   c.name as 班级,
7   sc.score as 分数
8 from
9   student s, course c, score sc
10 where
11   s.id = sc.student_id
12 and
13   c.id = sc.course_id
14 order by
15   s.id;
16
17 # 结果集中没有"不想毕业"同学的成绩, 因为score表中没有这位同学的记录
18 +-----+-----+-----+-----+-----+-----+
19 | id | 姓名      | 学号   | 性别 | 班级          | 分数    |
20 +-----+-----+-----+-----+-----+-----+
21 | 1 | 唐三藏    | 100001 | 1    | Java          | 70.5    |
22 | 1 | 唐三藏    | 100001 | 1    | MySQL         | 98.5    |
23 | 1 | 唐三藏    | 100001 | 1    | 计算机网络    | 33      |
24 | 1 | 唐三藏    | 100001 | 1    | 数据结构      | 98      |
25 | 2 | 孙悟空    | 100002 | 1    | Java          | 60      |
26 | 2 | 孙悟空    | 100002 | 1    | 计算机网络    | 59.5    |
27 | 3 | 猪悟能    | 100003 | 1    | Java          | 33      |
28 | 3 | 猪悟能    | 100003 | 1    | MySQL         | 68      |
29 | 3 | 猪悟能    | 100003 | 1    | 计算机网络    | 99      |
30 | 4 | 沙悟净    | 100004 | 1    | Java          | 67      |
31 | 4 | 沙悟净    | 100004 | 1    | MySQL         | 23      |
32 | 4 | 沙悟净    | 100004 | 1    | 计算机网络    | 56      |
33 | 4 | 沙悟净    | 100004 | 1    | 数据结构      | 72      |
34 | 5 | 宋江      | 200001 | 1    | Java          | 81      |
35 | 5 | 宋江      | 200001 | 1    | 计算机网络    | 37      |
36 | 6 | 武松      | 200002 | 1    | C++           | 56      |
37 | 6 | 武松      | 200002 | 1    | 操作系统      | 43      |
38 | 6 | 武松      | 200002 | 1    | 数据结构      | 79      |
39 | 7 | 李逵      | 200003 | 1    | C++           | 80      |
40 | 7 | 李逵      | 200003 | 1    | 数据结构      | 92      |
41 +-----+-----+-----+-----+-----+-----+
42 20 rows in set (0.00 sec)

```

4. 外连接

- 外连接分为**左外连接**、**右外连接**和**全外连接**三种类型，MySQL不支持全外连接。
- **左外连接**：返回左表的所有记录和右表中匹配的记录。如果右表中没有匹配的记录，则结果集中对应字段会显示为NULL。
- **右外连接**：与左外连接相反，返回右表的所有记录和左表中匹配的记录。如果左表中没有匹配的记录，则结果集中对应字段会显示为NULL。
- **全外连接**：结合了左外连接和右外连接的特点，返回左右表中的所有记录。如果某一边表中没有匹配的记录，则结果集中对应字段会显示为NULL。

4.1 语法

```

1 -- 左外连接，表1完全显示
2 select 字段名 from 表名1 left join 表名2 on 连接条件;
3 -- 右外连接，表2完全显示
4 select 字段 from 表名1 right join 表名2 on 连接条件;

```

4.2 示例

- 查询没有参加考试的同学信息

```

1 # 左连接以JOIN左边的表为基准，左表显示全部记录，右表中没有匹配的记录用NULL填充
2 mysql> select s.id, s.name, s.sno, s.age, sc.* from student s LEFT JOIN score
   sc on sc.student_id = s.id;
3 +-----+-----+-----+-----+-----+-----+-----+-----+
4 | id | name      | sno      | age | id | score | student_id | course_id |
5 +-----+-----+-----+-----+-----+-----+-----+-----+
6 | 1 | 唐三藏    | 100001   | 18  | 1 | 70.5 | 1 | 1 |
7 | 1 | 唐三藏    | 100001   | 18  | 2 | 98.5 | 1 | 3 |
8 | 1 | 唐三藏    | 100001   | 18  | 3 | 33   | 1 | 5 |
9 | 1 | 唐三藏    | 100001   | 18  | 4 | 98   | 1 | 6 |
10 | 2 | 孙悟空    | 100002   | 18  | 5 | 60   | 2 | 1 |
11 | 2 | 孙悟空    | 100002   | 18  | 6 | 59.5 | 2 | 5 |
12 | 3 | 猪悟能    | 100003   | 18  | 7 | 33   | 3 | 1 |
13 | 3 | 猪悟能    | 100003   | 18  | 8 | 68   | 3 | 3 |
14 | 3 | 猪悟能    | 100003   | 18  | 9 | 99   | 3 | 5 |
15 | 4 | 沙悟净    | 100004   | 18  | 10 | 67   | 4 | 1 |
16 | 4 | 沙悟净    | 100004   | 18  | 11 | 23   | 4 | 3 |
17 | 4 | 沙悟净    | 100004   | 18  | 12 | 56   | 4 | 5 |
18 | 4 | 沙悟净    | 100004   | 18  | 13 | 72   | 4 | 6 |
19 | 5 | 宋江      | 200001   | 18  | 14 | 81   | 5 | 1 |
20 | 5 | 宋江      | 200001   | 18  | 15 | 37   | 5 | 5 |
21 | 6 | 武松      | 200002   | 18  | 16 | 56   | 6 | 2 |
22 | 6 | 武松      | 200002   | 18  | 17 | 43   | 6 | 4 |

```

```

23 | 6 | 武松 | 200002 | 18 | 18 | 79 | 6 | 6 |
24 | 7 | 李逵 | 200003 | 18 | 19 | 80 | 7 | 2 |
25 | 7 | 李逵 | 200003 | 18 | 20 | 92 | 7 | 6 |
26 | 8 | 不想毕业 | 200004 | 18 | NULL | NULL | NULL | NULL |
27 +-----+-----+-----+-----+-----+-----+-----+
28 21 rows in set (0.00 sec)
29
30 # 过滤参加了考试的同学
31 mysql> select s.* from student s LEFT JOIN score sc on sc.student_id = s.id
      where sc.score is null;
32 +-----+-----+-----+-----+-----+-----+-----+
33 | id | name | sno | age | gender | enroll_date | class_id |
34 +-----+-----+-----+-----+-----+-----+-----+
35 | 8 | 不想毕业 | 200004 | 18 | 1 | 2000-09-01 | 1 |
36 +-----+-----+-----+-----+-----+-----+-----+
37 1 row in set (0.00 sec)

```

• 查询没有学生的班级

```

1 # 右连接以JOIN右边的表为基准，右表显示全部记录，左表中没有匹配的记录用NULL填充
2 mysql> select * from student s RIGHT JOIN class c on c.id = s.class_id;
3 +-----+-----+-----+-----+-----+-----+-----+-----+
4 | id | name | sno | age | gender | enroll_date | class_id | id |
5 | name |
6 +-----+-----+-----+-----+-----+-----+-----+-----+
7 | 4 | 沙悟净 | 100004 | 18 | 1 | 1986-09-01 | 1 | 1 |
8 | Java001班 |
9 | 3 | 猪八戒 | 100003 | 18 | 1 | 1986-09-01 | 1 | 1 |
10 | Java001班 |
11 | 2 | 孙悟空 | 100002 | 18 | 1 | 1986-09-01 | 1 | 1 |
12 | Java001班 |
13 | 1 | 唐三藏 | 100001 | 18 | 1 | 1986-09-01 | 1 | 1 |
14 | Java001班 |
15 | 8 | 不想毕业 | 200004 | 18 | 1 | 2000-09-01 | 2 | 2 |
16 | C++001班 |
17 | 7 | 李逵 | 200003 | 18 | 1 | 2000-09-01 | 2 | 2 |
18 | C++001班 |
19 | 6 | 武松 | 200002 | 18 | 1 | 2000-09-01 | 2 | 2 |
20 | C++001班 |
21 | 5 | 宋江 | 200001 | 18 | 1 | 2000-09-01 | 2 | 2 |
22 | C++001班 |
23 | NULL | NULL | NULL | NULL | NULL | NULL | NULL | 3 |
24 | 前端001班 |

```

```

15 +-----+-----+-----+-----+-----+-----+-----+-----+
16
17 # 过滤有学生的班级
18 mysql> select c.* from student s RIGHT JOIN class c on c.id = s.class_id where
    s.id is null;
19 +----+-----+
20 | id | name          |
21 +----+-----+
22 | 3  | 前端001班    |
23 +----+-----+
24 1 row in set (0.00 sec)

```

5. 自连接

5.1 应用场景

自连接是自己与自己取笛卡尔积，可以把行转化成列，在查询的时候可以使用where条件对结果进行过滤，或者说实现行与行之间的比较。在做表连接时为表起不同的别名。

```

1 # 不为表指定别名
2 mysql> select * from score, score;
3 ERROR 1066 (42000): Not unique table/alias: 'score'
4
5 # 指定别名
6 mysql> select * from score s1, score s2;
7

```

5.2 示例

- 显示所有"MySQL"成绩比"JAVA"成绩高的成绩信息

```

1 # 首先分两步进行，先查出JAVA和MySQL的课程Id，分别为1和3
2 mysql> select * from course where name = 'Java' or name = 'MySQL';
3 +----+-----+
4 | id | name  |
5 +----+-----+
6 | 1  | Java  |
7 | 3  | MySQL |
8 +----+-----+
9 2 rows in set (0.00 sec)
10

```

```

11 # 再查询成绩表中，JAVA成绩比MySQL成绩好的信息
12 mysql> select s1.* from score s1, score s2 where s1.student_id = s2.student_id
    and s1.course_id = 3 and s2.course_id = 1 and s1.score > s2.score;
13 +-----+-----+-----+-----+
14 | id | score | student_id | course_id |
15 +-----+-----+-----+-----+
16 | 2 | 98.5 | 1 | 3 |
17 | 8 | 68 | 3 | 3 |
18 +-----+-----+-----+-----+
19 2 rows in set (0.00 sec)
20
21 # 结合在一起进行查询
22 select s1.* from
23     score s1,
24     score s2,
25     course c1,
26     course c2
27 where
28     s1.student_id = s2.student_id
29 and
30     s1.course_id = c1.id
31 and
32     s2.course_id = c2.id
33 and
34     s1.score > s2.score
35 and
36     c1.`name` = 'MySQL'
37 and
38     c2.`name` = 'Java';
39
40 +-----+-----+-----+-----+
41 | id | score | student_id | course_id |
42 +-----+-----+-----+-----+
43 | 2 | 98.5 | 1 | 3 |
44 | 8 | 68 | 3 | 3 |
45 +-----+-----+-----+-----+
46 2 rows in set (0.00 sec)

```

5.3 表连接练习

- 显示所有"MySQL"成绩比"JAVA"成绩高的学生信息和班级以及成绩信息

```

1 # 相关的表全部加入连接，并确定连接条件
2 select stu.name as 姓名, c.name as 班级, s1.score as MySQL分数, s2.score as Java
   分数 from

```

```

3  score s1,
4  score s2,
5  course c1,
6  course c2,
7  student stu,
8  class c
9  where
10  s1.student_id = s2.student_id
11  and
12  s1.course_id = c1.id
13  and
14  s2.course_id = c2.id
15  and
16  s1.score > s2.score
17  and
18  stu.id = s1.student_id
19  and
20  stu.class_id = c.id
21  and
22  c1.`name` = 'MySQL'
23  and
24  c2.`name` = 'Java';
25
26 +-----+-----+-----+-----+
27 | 姓名      | 班级      | MySQL分数 | Java分数 |
28 +-----+-----+-----+-----+
29 | 唐三藏    | Java001班 | 98.5      | 70.5     |
30 | 猪悟能    | Java001班 | 68        | 33       |
31 +-----+-----+-----+-----+
32 2 rows in set (0.00 sec)

```

6. 子查询

子查询是把一个SELECT语句的结果当做别一个SELECT语句的条件，也叫嵌套查询

6.1 语法

```

1  select * from table1 where col_name1 {= | IN} (
2      select col_name1 from table2 where col_name2 {= | IN} [(
3          select ...)
4      ] ...
5  )

```

6.2 单行子查询

嵌套的查询中只返回一行数据

- 示例：查询与"不想毕业"同学的同班同学

```
1 mysql> select * from student where class_id = (select class_id from student
2         where name = '不想毕业');
3 +-----+-----+-----+-----+-----+-----+-----+
4 | id | name      | sno   | age | gender | enroll_date | class_id |
5 +-----+-----+-----+-----+-----+-----+-----+
6 | 5 | 宋江      | 200001 | 18 | 1      | 2000-09-01 | 2        |
7 | 6 | 武松      | 200002 | 18 | 1      | 2000-09-01 | 2        |
8 | 7 | 李逵      | 200003 | 18 | 1      | 2000-09-01 | 2        |
9 | 8 | 不想毕业  | 200004 | 18 | 1      | 2000-09-01 | 2        |
10 +-----+-----+-----+-----+-----+-----+-----+
11 4 rows in set (0.00 sec)
```

6.3 多行子查询

嵌套的查询中返回多行数据，使用[NOT] IN关键字

- 示例：查询"MySQL"或"Java"课程的成绩信息

```
1 mysql> select * from score where course_id in (select id from course where
2         name = 'Java' or name = 'MySQL');
3 +-----+-----+-----+-----+
4 | id | score | student_id | course_id |
5 +-----+-----+-----+-----+
6 | 1 | 70.5 | 1          | 1         |
7 | 5 | 60   | 2          | 1         |
8 | 7 | 33   | 3          | 1         |
9 | 10 | 67  | 4          | 1         |
10 | 14 | 81   | 5          | 1         |
11 | 2 | 98.5 | 1          | 3         |
12 | 8 | 68   | 3          | 3         |
13 | 11 | 23  | 4          | 3         |
14 +-----+-----+-----+-----+
15 8 rows in set (0.00 sec)
16 # 使用NOT IN 可以查询除了"MySQL"或"Java"课程的成绩
17 mysql> select * from score where course_id not in (select id from course where
18         name = 'Java' or name = 'MySQL');
19 +-----+-----+-----+-----+
20 | id | score | student_id | course_id |
```

```

20 +-----+-----+-----+-----+
21 | 3 | 33 | 1 | 5 |
22 | 4 | 98 | 1 | 6 |
23 | 6 | 59.5 | 2 | 5 |
24 | 9 | 99 | 3 | 5 |
25 | 12 | 56 | 4 | 5 |
26 | 13 | 72 | 4 | 6 |
27 | 15 | 37 | 5 | 5 |
28 | 16 | 56 | 6 | 2 |
29 | 17 | 43 | 6 | 4 |
30 | 18 | 79 | 6 | 6 |
31 | 19 | 80 | 7 | 2 |
32 | 20 | 92 | 7 | 6 |
33 +-----+-----+-----+-----+
34 12 rows in set (0.00 sec)

```

6.4 多列子查询

单行子查询和多行子查询都只返回一列数据，多列子查询中可以返回多个列的数据，外层查询与嵌套的内层查询的列要匹配

• 示例：查询重复录入的分数

```

1 # 插入重复的分数：score, student_id, course_id列重复
2 mysql> insert into score(score, student_id, course_id) values
3 (70.5, 1, 1),(98.5, 1, 3),(60, 2, 1);
4
5 # 子查询中返回多个列
6 mysql> SELECT * FROM score WHERE (score, student_id, course_id) IN (
7     SELECT score, student_id,
8     course_id FROM score GROUP BY score, student_id, course_id HAVING
9     count( 0 ) > 1
10 );
11
12 +-----+-----+-----+-----+
13 | id | score | student_id | course_id |
14 +-----+-----+-----+-----+
15 | 1 | 70.5 | 1 | 1 |
16 | 2 | 98.5 | 1 | 3 |
17 | 5 | 60 | 2 | 1 |
18 | 21 | 70.5 | 1 | 1 |
19 | 22 | 98.5 | 1 | 3 |
20 | 23 | 60 | 2 | 1 |
21 +-----+-----+-----+-----+
22 6 rows in set (0.00 sec)

```

6.5 在from子句中使用子查询

当一个查询产生结果时，MySQL自动创建一个临时表，然后把结果集放在这个临时表中，最终返回给用户，在from子句中也可以使用临时表进行子查询或表连接操作

• 示例：查询所有比"Java001班"平均分高的成绩信息

```
1 # 首先分步进行，第一步先查出Java001班的平均分
2 mysql> select avg(sc.score) score from student s
3         join class c on s.class_id = c.id
4         join score sc on s.id = sc.student_id
5         where
6           c.name = 'Java001班';
7 +-----+
8 | score |
9 +-----+
10 | 66.65625 |
11 +-----+
12 1 row in set (0.00 sec)
13
14 # 把以上查询做为临时表，与真实表进行比较
15 mysql> select * from score s, (
16         select avg(sc.score) score from student s
17         join class c on s.class_id = c.id
18         join score sc on s.id = sc.student_id
19         where
20           c.name = 'Java001班') tmp
21         where s.score > tmp.score;
22 +-----+-----+-----+-----+-----+
23 | id | score | student_id | course_id | score |
24 +-----+-----+-----+-----+-----+
25 | 1 | 70.5 | 1 | 1 | 66.65625 |
26 | 2 | 98.5 | 1 | 3 | 66.65625 |
27 | 4 | 98 | 1 | 6 | 66.65625 |
28 | 8 | 68 | 3 | 3 | 66.65625 |
29 | 9 | 99 | 3 | 5 | 66.65625 |
30 | 10 | 67 | 4 | 1 | 66.65625 |
31 | 13 | 72 | 4 | 6 | 66.65625 |
32 | 14 | 81 | 5 | 1 | 66.65625 |
33 | 18 | 79 | 6 | 6 | 66.65625 |
34 | 19 | 80 | 7 | 2 | 66.65625 |
35 | 20 | 92 | 7 | 6 | 66.65625 |
36 | 21 | 70.5 | 1 | 1 | 66.65625 |
37 | 22 | 98.5 | 1 | 3 | 66.65625 |
38 +-----+-----+-----+-----+-----+
```

```
39 13 rows in set (0.00 sec)
40
```

tmp 是临时表的别名

7. 合并查询

在实际应用中，为了合并多个select操作返回的结果，可以使用集合操作符 union，union all

7.1 创建新表并初始化数据

```
1 # 创建一个新表并初始化数据
2 mysql> create table student1 like student;
3 Query OK, 0 rows affected (0.03 sec)
4
5 insert into student1 (name, sno, age, gender, enroll_date, class_id) values
6 ('唐三藏', '100001', 18, 1, '1986-09-01', 1),
7 ('刘备', '300001', 18, 1, '1993-09-01', 3),
8 ('张飞', '300002', 18, 1, '1993-09-01', 3),
9 ('关羽', '300003', 18, 1, '1993-09-01', 3);
10
11 Query OK, 4 rows affected (0.01 sec)
12 Records: 4 Duplicates: 0 Warnings:
13
14 mysql> select * from student1;
15 +-----+-----+-----+-----+-----+-----+-----+
16 | id | name   | sno   | age | gender | enroll_date | class_id |
17 +-----+-----+-----+-----+-----+-----+-----+
18 | 1 | 唐三藏 | 100001 | 18 | 1 | 1986-09-01 | 1 |
19 | 2 | 刘备   | 300001 | 18 | 1 | 1993-09-01 | 3 |
20 | 3 | 张飞   | 300002 | 18 | 1 | 1993-09-01 | 3 |
21 | 4 | 关羽   | 300003 | 18 | 1 | 1993-09-01 | 3 |
22 +-----+-----+-----+-----+-----+-----+-----+
23 4 rows in set (0.00 sec)
```

7.2 Union

该操作符用于取得两个结果集的并集。当使用该操作符时，会自动去掉结果集中的重复行。

- 示例：查询student表中 id < 3 的同学和student1表中的所有同学

```
1 # 结果集中有两张表中的数据，但是唐三藏只返回了一条记录
2 mysql> select * from student where id < 3 union select * from student1;
```

```

3 +-----+-----+-----+-----+-----+-----+-----+
4 | id | name      | sno   | age  | gender | enroll_date | class_id |
5 +-----+-----+-----+-----+-----+-----+-----+
6 | 1 | 唐三藏    | 100001 | 18   | 1      | 1986-09-01  | 1        |
7 | 2 | 孙悟空    | 100002 | 18   | 1      | 1986-09-01  | 1        |
8 | 2 | 刘备      | 300001 | 18   | 1      | 1993-09-01  | 3        |
9 | 3 | 张飞      | 300002 | 18   | 1      | 1993-09-01  | 3        |
10 | 4 | 关羽      | 300003 | 18   | 1      | 1993-09-01  | 3        |
11 +-----+-----+-----+-----+-----+-----+-----+
12 5 rows in set (0.00 sec)

```

7.3 Union all

该操作符用于取得两个结果集的并集。当使用该操作符时，不会去掉结果集中的重复行。

```

1 # 结果集中有两张表中的数据，返回了所有唐三藏的记录
2 mysql> select * from student where id < 3 union all select * from student1;
3 +-----+-----+-----+-----+-----+-----+-----+
4 | id | name      | sno   | age  | gender | enroll_date | class_id |
5 +-----+-----+-----+-----+-----+-----+-----+
6 | 1 | 唐三藏    | 100001 | 18   | 1      | 1986-09-01  | 1        |
7 | 2 | 孙悟空    | 100002 | 18   | 1      | 1986-09-01  | 1        |
8 | 1 | 唐三藏    | 100001 | 18   | 1      | 1986-09-01  | 1        |
9 | 2 | 刘备      | 300001 | 18   | 1      | 1993-09-01  | 3        |
10 | 3 | 张飞      | 300002 | 18   | 1      | 1993-09-01  | 3        |
11 | 4 | 关羽      | 300003 | 18   | 1      | 1993-09-01  | 3        |
12 +-----+-----+-----+-----+-----+-----+-----+
13 6 rows in set (0.00 sec)

```

8. 插入查询结果

8.1 语法

```

1 INSERT INTO table_name [(column [, column ...])] SELECT ...

```

8.2 示例

- 将student表中C++001班的学生复制到student1表中

```

1 mysql> insert into student1 (name, sno, age, gender, enroll_date, class_id)
2     select s.name, s.sno, s.age, s.gender, s.enroll_date, s.class_id
3     from student s, class c where s.class_id = c.id and c.name = 'C++001班';
4 Query OK, 4 rows affected (0.01 sec)
5 Records: 4  Duplicates: 0  Warnings: 0
6
7 mysql> select * from student1;
8 +-----+-----+-----+-----+-----+-----+-----+
9 | id | name      | sno    | age | gender | enroll_date | class_id |
10 +-----+-----+-----+-----+-----+-----+-----+
11 | 1 | 唐三藏    | 100001 | 18  | 1      | 1986-09-01  | 1        |
12 | 2 | 刘备      | 300001 | 18  | 1      | 1993-09-01  | 3        |
13 | 3 | 张飞      | 300002 | 18  | 1      | 1993-09-01  | 3        |
14 | 4 | 关羽      | 300003 | 18  | 1      | 1993-09-01  | 3        |
15 | 5 | 宋江      | 200001 | 18  | 1      | 2000-09-01  | 2        |
16 | 6 | 武松      | 200002 | 18  | 1      | 2000-09-01  | 2        |
17 | 7 | 李逵      | 200003 | 18  | 1      | 2000-09-01  | 2        |
18 | 8 | 不想毕业  | 200004 | 18  | 1      | 2000-09-01  | 2        |
19 +-----+-----+-----+-----+-----+-----+-----+
20 8 rows in set (0.00 sec)

```